



Learning Without Forgetting for Continual Learning in LLMs Through Adaptive LoRA Ranks

Fuli Qiao¹ · Mehrdad Mahdavi¹

Received: 26 October 2025 / Revised: 13 July 2026 / Accepted: 11 August 2026

© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2026

Abstract

Large language models (LLMs) have achieved remarkable performance across a wide range of downstream tasks, with parameter-efficient fine-tuning methods such as LoRA significantly reducing computational overhead. However, LLMs remain vulnerable to catastrophic forgetting when learning sequential tasks, and the rank of LoRA is typically fixed at the same value across all model layers and tasks, leaving the role of rank underexplored in continual learning. In this paper, we propose a novel parameter-efficient approach for continual learning in LLMs, which adaptively adjusts layerwise LoRA ranks to help balance the stability–plasticity trade-off. Our proposed method leverages a subspace similarity metric to evaluate the orthogonality between task-specific low-rank subspaces and thus adaptively increases the layerwise rank for each new task, thereby minimizing interference with previously learned tasks while enhancing generalization. Experimental results on standard natural language processing continual learning benchmarks and challenging mathematical benchmarks show that our method achieves comparable or superior performance in mitigating forgetting, improving task performance, and maintaining strong generalization to unseen data in a memory-efficient manner. Code is provided in the [link](#).

Keywords Continual learning · LLMs · Parameter-efficient · LoRA · Low-Rank

Editor: Enver Sangineto.

✉ Mehrdad Mahdavi
mzm616@psu.edu

Fuli Qiao
fvq5015@psu.edu

¹ Department of Computer Science and Engineering, Pennsylvania State University, State College, PA 16801, USA

1 Introduction

The rapid scaling of Large Language Models (LLMs) (Raffel et al., 2020; Chowdhery et al., 2023; Achiam et al., 2023; Touvron et al., 2023) has revolutionized their adaptation to numerous downstream static tasks through parameter-efficient fine-tuning (PEFT), like LoRA (Hu et al., 2022) and its multiple variants (Zhang et al., 2023; Liu et al., 2024). However, a critical challenge persists in continual learning (CL) (Wu et al., 2021, 2024), which is the ability to learn multiple sequential tasks. There are two key obstacles: (i) catastrophic forgetting, where a model's performance on earlier tasks degrades when trained on new tasks (McCloskey & Cohen, 1989; Ratcliff, 1990), and (ii) generalization ability, where the previously learned model can improve the learning of new tasks (Carpenter & Grossberg, 1987). In continual learning, these two obstacles are inherently coupled through a stability–plasticity trade-off: overly strong constraints to prevent forgetting can limit the model's capacity to generalize to new tasks, while aggressive adaptation to new tasks can exacerbate forgetting. Understanding the trade-off between forgetting and generalization is the key to understanding the effectiveness of the regularization-based CL methods (Carpenter & Grossberg, 1987; Grossberg, 2013; Li et al., 2023).

Although existing LoRA-based PEFT methods for CL have mitigated the forgetting issue, such as O-LoRA (Wang et al., 2023) that incrementally learns new tasks in orthogonal subspaces, most approaches apply the same rank across all layers of the model, disregarding the heterogeneous nature of different layers in overparameterized models, which have been extensively studied as highlighted in (Zhang et al., 2022). Recent rank-aware methods in pre-training and adaptation for LLMs have highlighted the role of rank. For instance, AdaRank (Dong, 2024) introduces a simple model disagreement-based technique for determining layerwise ranks for low-rank adaptation induced by random module perturbations. Similarly, AdaLoRA (Zhang et al., 2023) and SoRA (Ding et al., 2023) exploit the relationship between the rank and the singular value decomposition of the weight update matrices to dynamically adjust layerwise ranks during adaptation. Specifically, AdaLoRA prunes singular values associated with less significant updates, while SoRA employs a learnable gating mechanism to gradually reduce rank during training. These findings strongly suggest that using varying ranks for different layers is more effective, as enforcing the same rank across all layers may lead to overfitting certain features and diminished generalization.

Recent studies have shown that LoRA forgets less than full fine-tuning (Biderman et al., 2024) and common regularization techniques like weight decay and dropout (Hyder et al., 2022; Biderman et al., 2024), and also helps maintain diversity in generations. However, the low-rank update mechanism limits the ability of LLMs to learn and retain new knowledge as effectively as full fine-tuning (Hu et al., 2022; Xia et al., 2024; Hao et al., 2024; Zhao et al., 2024), especially in challenging tasks like mathematical reasoning. To address this limitation, several recent works investigated the dynamics of LoRA to improve the performance (Xia et al., 2024; Hao et al., 2024). COLA (Xia et al., 2024) employs iterative low-rank residual learning to approximate optimal weight updates for task adaptation, resulting in increasing ranks of LoRAs by extending the length of the chain, and FLORA (Hao et al., 2024) achieves high-rank updates by resampling projection matrices. While rank dynamics have been explored in static fine-tuning tasks, the impact of rank patterns has not been thoroughly examined in CL for LLMs for mitigating forgetting and enhancing generalization.

To examine the impact of layerwise ranks in incremental learning of LoRA between tasks for CL, our pilot experiment uses a fixed uniform rank across all layers, testing two different rank settings and freezing previously learned incremental LoRAs without regularization when training new tasks. The results, shown in Fig. 1 which uses pre-trained T5-large model (Raffel et al., 2020) with fixed-rank incremental LoRAs for DBpedia and Amazon Reviews (Zhang et al., 2015), indicate that while similar average accuracy across tasks can be achieved with different low ranks, in this special case, a rank of 8 for the second task tends to cause greater accuracy loss on the first task, and a rank of 4 for both tasks cannot achieve the same level of performance on the current task as with rank 8. Importantly, this does not imply a direct linear relationship between the size of the rank and the degree of forgetting. Instead, it reveals a trade-off between different ranks to balance forgetting mitigation and generalization. These observations highlight the need for an ideal approach in CL for LLMs within the PET framework, one that utilizes the role of the layerwise ranks to balance catastrophic forgetting and generalization across a sequential stream of tasks. Inspired by this, we aim to address the following fundamental question:

How can an adaptive, parameter-efficient CL algorithm leverage low ranks to balance the trade-off between mitigating forgetting and enhancing generalization?

To answer this question, we propose CL-Rank, a novel PEFT-CL algorithm that adaptively optimizes layerwise low-rank subspaces through inter-task low-rank subspace similarity analysis during training among layers. Our key insight is that optimal rank distribution varies across both layer LoRAs and task sequences in CL. Specifically, CL-Rank implements this via the following mechanisms: it initializes each new sequential task's incremental LoRA with a rank of 1. During training, it periodically computes a layer-wise low-rank subspace metric by extracting left singular vectors of LoRAs from both the current and previous tasks. Based on the measured subspace orthogonality, the method then decides whether to increase the rank via adding new low-rank parameters when the similarity is in the range of the threshold, or to maintain the current rank, thereby enforcing conservative updates that limit interference with previous tasks. This mechanism is designed to promote a better trade-off in mitigating forgetting and improving generalization. Note that CL-Rank keeps all previously learned incremental LoRAs frozen when training a new task, and the mechanism is activated starting from the second task. Our experimental results demon-

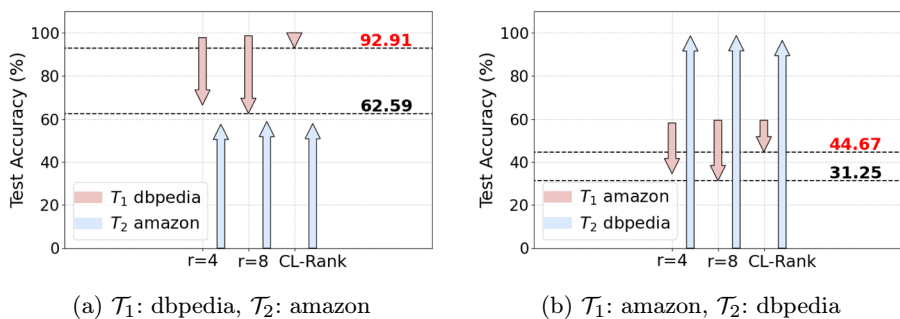


Fig. 1 Comparison of each task accuracy changes after training second task T_2 under two task orders and LoRA ranks (4 and 8) using incremental LoRA fine-tuning: **a** fine-tune T_1 dbpedia using a LoRA, then add a new LoRA to fine-tune second task T_2 amazon with the first LoRA frozen, **b** fine-tune T_1 amazon using a LoRA, then add a new LoRA to fine-tune second task T_2 dbpedia with the first LoRA frozen

strate that CL-Rank has comparable or superior performance on standard continual learning benchmarks and excels in more challenging math tasks, such as GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021). Furthermore, our analysis explores the impact of various hyperparameters and evaluates different rank updating rules, highlighting the effectiveness of CL-Rank in robustness to task orders, mitigating forgetting, and enhancing generalization.

This paper makes three key contributions: (1) A novel parameter-efficient continual learning method for LLMs that effectively balances forgetting and generalization through adaptive low-rank updates; (2) Through comprehensive evaluations, our method achieves comparable or superior performance over existing approaches applicable to two LLMs architectures (T5 and Llama-2) both on standard continual learning benchmarks and math datasets; and (3) We provide an in-depth analysis that deepens our understanding of the dynamics of adaptively increasing rank within continual learning for LLMs, pinpointing critical factors that drive its effectiveness.

2 Related Works

2.1 Continual Learning

Continual learning aims to develop algorithms that can continuously accumulate and refine knowledge, especially when handling dynamic data streams. The key challenge is overcoming catastrophic forgetting, where a model's performance on previously learned tasks significantly declines after being trained on new tasks. To tackle this issue, existing approaches are generally classified into three main categories: (i) *Rehearsal-based methods*, which use a memory buffer to retain data samples from previous tasks, incorporating techniques such as experience replay (Rolnick et al., 2019), or constrained optimization to allow the model to learn from current and previous tasks simultaneously (Lopez-Paz & Ranzato, 2017; Han et al., 2020). (ii) *Regularization-based methods*, which add extra terms to the loss function to penalize changes in important model parameters, limiting interference with previously learned tasks (Kirkpatrick et al., 2017; Li & Hoiem 2017; Farajtabar et al., 2020; Smith et al., 2023). For example, EWC (Kirkpatrick et al., 2017) preserves knowledge of old tasks by slowing down learning on weights deemed important for those tasks, while OGD (Farajtabar et al., 2020) ensures that parameters move within the orthogonal space defined by previous task gradients. (iii) *Architecture-based methods*, which aim to reduce task interference by dynamically expanding the model's capacity or creating separate components for each task (Rusu et al., 2016; Yoon et al., 2017; Li et al., 2019; Rao et al., 2019; Razdaibiedina et al., 2023; Jia et al., 2025; Lu et al., 2024; Zhou et al., 2024). For instance, Progressive Prompts (Razdaibiedina et al. 2023) improves forward transfer and mitigates forgetting by learning a distinct prompt for each new task and sequentially appending these task-specific prompts to previously learned ones. NSP² (Lu et al., 2024) uses SVD to find a null-space projection for the new task to update during training, not interfering with previous tasks. The SVD is applied to covariance matrices derived from prompt representations, and the resulting projection is used to project gradients or updates during training. EASE (Zhou et al., 2024) allocates task-specific adapters in a class-incremental learning setting and does not perform geometric analysis of parameter-update subspaces via SVD.

2.2 Parameter-Efficient Tuning

Recent works on parameter-efficient tuning (PET) (He et al., 2021) have demonstrated that training only a subset of model parameters can achieve performance comparable to full model fine-tuning, while significantly reducing computational and annotation costs (Houlsby et al., 2019; Zaken et al., 2021; Lester et al., 2021; Hu et al., 2022; Zhang et al., 2023). For instance, BitFit (Zaken et al. 2021) shows that updating only the bias terms during fine-tuning is highly effective. Prompt tuning (Lester et al. 2021) leverages learnable 'soft prompts' via back-propagation to condition frozen language models for specific tasks. LoRA (Hu et al., 2022) employs low-rank adapters to adapt models to new tasks with minimal additional parameters, and AdaLoRA (Zhang et al., 2023) builds on LoRA by dynamically allocating the parameter budget based on the importance of the weight matrices. While most PET methods focus on learning a single task, some efforts have extended PET to continual learning. AdapterCL (Madotto et al., 2020) introduces a dedicated adapter block for each task, and LFPT5 (Qin & Joty, 2021) continuously trains a large soft prompt across multiple tasks. ConPET (Song et al., 2023) adapts existing continual learning strategies-originally developed for smaller models-to LLMs by integrating PET with a dynamic replay mechanism. MoILE (Jia et al., 2025) applies SVD to a router-selected LoRA expert and divides its parameters into principal component parameters and residual component parameters. It freezes principal component parameters so as not to interfere with previously trained tasks and updates only the residual component parameters during training a new task. SAPT-LoRA (Zhao et al., 2024) aligns the PET learning and selection via the shared module, but the limitation is that they heavily depend on previous tasks' data samples when training new tasks. O-LoRA (Wang et al., 2023) incrementally learns new tasks in orthogonal subspaces, keeping LoRA parameters from previous tasks fixed to mitigate catastrophic forgetting. However, O-LoRA uses the same fixed rank for all incremental LoRAs without investigating the rank patterns in CL.

3 Adaptive Low-Rank Continual Learner

In this section, we propose a parameter-efficient continual learning approach that dynamically optimizes low-rank subspaces to balance forgetting mitigation via low-rank constraints and generalization enhancement via rank adaptation.

3.1 Problem Setting

In continual learning, we have a sequence of tasks $\mathcal{T} = \{\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_N\}$ over time. Each task $\mathcal{T}_i$ has a data distribution $\mathcal{D}_i$ and a separate target dataset $\mathcal{S}_i = \{(\mathbf{x}_{i,k}, y_{i,k})\}_{k=1}^{n_i}$ where $\mathbf{x}_{i,k} \in \mathcal{X}_i$ and $y_{i,k} \in \mathcal{Y}_i$. The goal of continual learning is to find a set of parameters $\theta \in \Theta$ that effectively solves all tasks up to the current task $\mathcal{T}_i$, while minimizing catastrophic forgetting of previously learned tasks. In continual learning of LLMs, we are given a pre-trained model $\mathbf{W}_0$ and continually fine-tune a sequence of tasks, utilizing incremental low-rank matrices $\mathbf{B}_i \mathbf{A}_i$ to fine-tune task $\mathcal{T}_i$ while freezing prior tasks' LoRAs, where $\mathbf{B}_i \in \mathbb{R}^{d_1 \times r_i}$, $\mathbf{A}_i \in \mathbb{R}^{r_i \times d_2}$ and the rank $r_i \ll \min(d_1, d_2)$. The continual learning model

parameters after fine-tuning task $\mathcal{T}_i$ is $\theta_i = \mathbf{W}_0 + \sum_{s=1}^i \mathbf{B}_s \mathbf{A}_s$. Our continual learning goal is to optimize the following objective across all tasks:

$$\max_{\theta} \sum_{i=1}^N \sum_{(\mathbf{x}, y) \in \mathcal{S}_i} \log p_{\theta}(y|\mathbf{x}), \quad (1)$$

where $\theta = \mathbf{W}_0 + \sum_{i=1}^N \mathbf{B}_i \mathbf{A}_i$. It is important to note that in our scenario, the model does not have access to data from previous tasks when learning a new task, and the model predicts sample labels without task IDs during testing.

3.2 Forgetting Error in Low-Rank CL

In this section, we examine the forgetting error to illustrate the proposed method. This analysis aims to elucidate the key idea to gradually increase the rank to mitigate forgetting. Inspired by Li et al. (2023), we consider a simple linear regression setting with two tasks. Each task $\mathcal{T}_k$ is associated with a data distribution $\mathcal{D}_k$ and contains a separate target dataset $\mathcal{S}_k = \{(\mathbf{x}_{k,i}, y_{k,i})\}_{i=1}^{n_k}$ where $\mathbf{x}_{k,i} \in \mathcal{X}_k$ and $y_{k,i} \in \mathcal{Y}_k$. For simplicity, we use two tasks and all $n_k = n$ for explanation. The population risks for the two tasks can be denoted by

$$\mathcal{R}_1(\mathbf{W}_0 + \mathbf{B}\mathbf{A}) = \frac{1}{n} \mathbb{E}_{\mathcal{D}_1} \|y_1 - \mathbf{X}_1(\mathbf{W}_0 + \mathbf{B}\mathbf{A})\|^2 \quad (2)$$

$$\mathcal{R}_2(\mathbf{W}_0 + \mathbf{B}\mathbf{A}) = \frac{1}{n} \mathbb{E}_{\mathcal{D}_2} \|y_2 - \mathbf{X}_2(\mathbf{W}_0 + \mathbf{B}\mathbf{A})\|^2 \quad (3)$$

respectively, where $(\mathbf{X}_1, y_1)$ is the dataset of the first task and $(\mathbf{X}_2, y_2)$ is the dataset of the second task. $\mathbf{X}_k = (\mathbf{x}_{k,1}, \dots, \mathbf{x}_{k,n})^\top \in \mathbb{R}^{n \times d}$ and $y_K = (y_{k,1}, \dots, y_{k,n}) \in \mathbb{R}^n$ for $k = 1, 2$.

Assumption 1 (Fixed design) Assume that the feature vectors $(\mathbf{x}_{1,i})_{i=1}^n$ and $(\mathbf{x}_{2,i})_{i=1}^n$ are fixed and that the labels $(y_{1,i})_{i=1}^n$ and $(y_{2,i})_{i=1}^n$ are independent random variables.

Assumption 2 (Shared optimal parameter) Assume that there exists a $\mathbf{B}^* \mathbf{A}^*$ where $\mathbf{B}^* \in \mathbb{R}^{d_1 \times r}$, $\mathbf{A}^* \in \mathbb{R}^{r \times d_2}$ such that

$$\mathbf{B}^* \mathbf{A}^* \in \arg \min \mathcal{R}_1(\mathbf{W}_0 + \mathbf{B}\mathbf{A}), \quad \mathbf{B}^* \mathbf{A}^* \in \arg \min \mathcal{R}_2(\mathbf{W}_0 + \mathbf{B}\mathbf{A}) \quad (4)$$

We assume there is a common optimal low-rank parameter for the two tasks, which follows (Li et al., 2023; Ven & Toliás, 2019; Evron et al., 2022).

Assumption 3 (Well-specified noise) Assume that for $\mathbf{B}^* \mathbf{A}^*$ in Assumption 2, it holds that: for $k = 1, 2$ and $i = 1, \dots, n$,

$$\mathbb{E}[y_{k,i}] = \mathbf{x}_{k,i}^\top (\mathbf{W}_0 + \mathbf{B}^* \mathbf{A}^*) \quad (5)$$

$$\sigma^2 = \mathbb{E}(y_{k,i} - \mathbf{x}_{k,i}^\top (\mathbf{W}_0 + \mathbf{B}^* \mathbf{A}^*))^2 \quad (6)$$

where $\sigma^2 > 0$ refers to the variance of the label noise.

Assumption 4 (Commutable data covariance matrices) Assume that

$$\mathbf{H}_1 \mathbf{H}_2 = \mathbf{H}_2 \mathbf{H}_1, \quad \text{where } \mathbf{H}_1 = \frac{1}{n} \mathbf{X}_1^\top \mathbf{X}_1 \text{ and } \mathbf{H}_2 = \frac{1}{n} \mathbf{X}_2^\top \mathbf{X}_2 \quad (7)$$

Risk decomposition. For simplicity, we use $\delta_1 = \mathbf{B}_1 \mathbf{A}_1$, $\delta_2 = \mathbf{B}_2 \mathbf{A}_2$, and $\delta^* = \mathbf{B}^* \mathbf{A}^*$. According to the risk definition and the assumptions on the noise, we have

$$\begin{aligned} \mathcal{R}_1(\mathbf{W}_0 + \delta) &= \frac{1}{n_1} \mathbb{E} \|\mathbf{X}_1(\mathbf{W}_0 + \delta_1) - y_1\|^2 \\ &= \frac{1}{n_1} \mathbb{E} \|\mathbf{X}_1(\mathbf{W}_0 + \delta_1) - \mathbf{X}_1(\mathbf{W}_0 + \delta^*) - \epsilon_1\|^2 \\ &= (\mathbf{W}_0 + \delta^*)^\top \mathbf{H}_1 (\mathbf{W}_0 + \delta^*) + \sigma^2 \\ &= \langle \mathbf{H}_1, (\mathbf{W}_0 + \delta^*)(\mathbf{W}_0 + \delta^*)^\top \rangle + \sigma^2 \end{aligned} \quad (8)$$

Similarly, the risk for the second task is

$$\mathcal{R}_2(\mathbf{W}_0 + \delta) = \langle \mathbf{H}_2, (\mathbf{W}_0 + \delta^*)(\mathbf{W}_0 + \delta^*)^\top \rangle + \sigma^2 \quad (9)$$

Computing forgetting. We now compute forgetting according to Eqs. 8 and 9.

$$\begin{aligned} \mathcal{F} &= \mathbb{E}[\mathcal{R}_1(\mathbf{W}_0 + \delta_1 + \delta_2) - \mathcal{R}_1(\mathbf{W}_0 + \delta_1)] \\ &= \langle \mathbf{H}_1, \mathbb{E}(\mathbf{W}_0 + \delta_1 + \delta_2 - (\mathbf{W}_0 + \delta^*))(\mathbf{W}_0 + \delta_1 + \delta_2 - (\mathbf{W}_0 + \delta^*))^\top \rangle \\ &\quad - \langle \mathbf{H}_1, \mathbb{E}(\mathbf{W}_0 + \delta_1 - (\mathbf{W}_0 + \delta^*))(\mathbf{W}_0 + \delta_1 - (\mathbf{W}_0 + \delta^*))^\top \rangle \\ &= \langle \mathbf{H}_1, \mathbb{E}(\delta_1 + \delta_2 - \delta^*)(\delta_1 + \delta_2 - \delta^*)^\top \rangle - \langle \mathbf{H}_1, \mathbb{E}(\delta_1 - \delta^*)(\delta_1 - \delta^*)^\top \rangle \\ &= \langle \mathbf{H}_1, \mathbb{E}(\delta_1 \delta_1^\top + \delta_1 \delta_2^\top - \delta_1 (\delta^*)^\top + \delta_2 \delta_1^\top + \delta_2 \delta_2^\top - \delta_2 (\delta^*)^\top - \delta^* \delta_1^\top - \delta^* \delta_2^\top \\ &\quad + \delta^* (\delta^*)^\top) \rangle - \langle \mathbf{H}_1, \mathbb{E}(\delta_1 \delta_1^\top - \delta_1 (\delta^*)^\top - \delta^* \delta_1^\top + \delta^* (\delta^*)^\top) \rangle \\ &= \langle \mathbf{H}_1, \mathbb{E}(\delta_1 \delta_2^\top + \delta_2 \delta_1^\top + \delta_2 \delta_2^\top - \delta_2 (\delta^*)^\top - \delta^* \delta_2^\top) \rangle \\ &\stackrel{(a)}{=} \& \text{tr}(\mathbf{H}_1^\top \mathbb{E}(\delta_1 \delta_2^\top + \delta_2 \delta_1^\top + \delta_2 \delta_2^\top - \delta_2 (\delta^*)^\top - \delta^* \delta_2^\top)) \\ &\stackrel{(b)}{\leq} \& \|\mathbf{H}_1\| \text{tr}(\mathbb{E}(\delta_1 \delta_2^\top + \delta_2 \delta_1^\top + \delta_2 \delta_2^\top - \delta_2 (\delta^*)^\top - \delta^* \delta_2^\top)) \\ &\stackrel{(c)}{=} \& \|\mathbf{H}_1\| (2\text{tr}(\mathbb{E} \delta_1 \delta_2^\top) + \text{tr}(\mathbb{E} \delta_2 \delta_2^\top) - 2\text{tr}(\mathbb{E} \delta_2 (\delta^*)^\top)) \\ &\stackrel{(d)}{\leq} \& \|\mathbf{H}_1\| (2\text{tr}(\mathbb{E} \delta_1 \delta_2^\top) + r \|\mathbb{E} \delta_2\|_2^2 - 2\text{tr}(\mathbb{E} \delta_2 (\delta^*)^\top)) \end{aligned} \quad (10)$$

where r is the rank of δ_2 . To explain the derivation steps clearly, we assume that $\mathbf{A}$ and $\mathbf{B}$ are two matrices, step (a) is from $\langle \mathbf{A}, \mathbf{B} \rangle = \text{tr}(\mathbf{B}^\top \mathbf{A})$, step (b) is from $\text{tr}(\mathbf{A}^\top \mathbf{B}) \leq \|\mathbf{A}\| \text{tr}(\mathbf{B})$, step (c) is from $\text{tr}(\mathbf{A} + \mathbf{B}) = \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B})$, step (d) is obtained from $\text{tr}(\delta_2 \delta_2^\top) \leq r \|\delta_2\|_2^2$.

Analysis of the upper bound of $\mathcal{F}$.

- The matrix $\mathbf{H}_1$ is a data correlation matrix, and in practice, we cannot directly control its magnitude.
- The term $r \|\mathbb{E} \delta_2\|_2^2$ becomes smaller when using a lower rank for δ_2 , which helps in reducing the overall forgetting error.
- Additionally, $\text{tr}(\mathbb{E} \delta_2 (\delta^*)^\top)$ is also expected to be small when using a low-rank ap-

proximation for δ_2 .

Therefore, the upper bound on the forgetting error is dominated by the term involving $\delta_1 \delta_2^\top$, denoted by:

$$\mathcal{F} \leq \mathcal{O}(\text{tr}(\delta_1 \delta_2^\top)) \quad (11)$$

This finding motivates us to find an effective subspace similarity between tasks during training to control and optimize the forgetting and generalization trade-off. Our analysis is intentionally lightweight and focuses on the interaction between task-specific low-rank subspaces, which directly aligns with the structural role of LoRA. In LoRA-based fine-tuning, updates are linear in the low-rank parameters and are applied additively to frozen backbone weights. Under standard linearization assumptions commonly used in prior works on PEFT and NTK-style analyses, interference between tasks is primarily governed by the overlap between their update subspaces. From this perspective, subspace similarity serves as a principled proxy for measuring potential interference, even if the underlying model is nonlinear.

3.3 Inter-Task Layerwise Low-Rank Subspace Similarity Measure

To measure the low-rank subspace similarity between different tasks, we utilize a reverse metric (Hu et al., 2022) of the standard Projection Metric of Grassmann Distance that measures the distance between subspaces (Hamm & Lee, 2008). For tasks $\mathcal{T}_i$ and $\mathcal{T}_j$, we define the low-rank subspace at layer l as $B_i^l A_i^l$ and $B_j^l A_j^l$, where $B_i^l \in \mathbb{R}^{d_1 \times r_i^l}$ and $A_i^l \in \mathbb{R}^{r_i^l \times d_2}$, and similarly for $\mathcal{T}_j$. Taking SVD of $B_i^l A_i^l$ and $B_j^l A_j^l$, we obtain their respective top r_i^l and top r_j^l left singular vectors: $U_i^l \in \mathbb{R}^{d_1 \times r_i^l}$ and $U_j^l \in \mathbb{R}^{d_1 \times r_j^l}$. Then we let the singular values of $(U_i^l)^\top U_j^l$ be $\sigma_1, \sigma_2, \dots, \sigma_p$, where $p = \min\{r_i^l, r_j^l\}$. The Grassmann Distance standard projection metric is defined as: $d(U_i^l, U_j^l) = \sqrt{p - \sum_{s=1}^p \sigma_s^2} \in [0, \sqrt{p}]$. Following the Grassmann Distance, we define our task subspace similarity metric as:

$$\phi(U_i^l, U_j^l) = \frac{\sum_{s=1}^p \sigma_s^2}{p} = 1 - \frac{d(U_i^l, U_j^l)^2}{p} \quad (12)$$

This similarity metric satisfies the following conditions: when U_i^l and U_j^l share the same column span, considered as overlapping, then $\phi(U_i^l, U_j^l) = 1$. If they are completely orthogonal, then $\phi(U_i^l, U_j^l) = 0$. Otherwise, $\phi(U_i^l, U_j^l) \in (0, 1)$. We use this metric to determine whether the low-rank subspaces of two tasks are orthogonal. If $\phi(U_i^l, U_j^l) < \epsilon$, the low-rank subspaces of tasks $\mathcal{T}_i$ and $\mathcal{T}_j$ are orthogonal, meaning increasing the rank in the current subspace is “safe” for both tasks, as it would not interfere with the learned low-rank subspaces of previous tasks. Conversely, if $\phi(U_i^l, U_j^l) > \epsilon$, which means the subspaces are not orthogonal, we maintain the current rank for the new task to reduce the risk of forgetting prior tasks. Regarding the choice of similarity metric, we emphasize that our intent is not to claim optimality, but sufficiency and robustness. The proposed similarity measure captures the degree of alignment between task-specific LoRA update directions, which is precisely the quantity implicated in our forgetting upper bound analysis.

3.4 Insights on Subspace Orthogonality

Here, we analyze the relationship between subspace inner products and singular values. For two tasks $\mathcal{T}_i$ and $\mathcal{T}_j$, their low-rank subspaces capture task-specific directions of gradient updates. When $\langle \mathbf{B}_i^l \mathbf{A}_i^l, \mathbf{B}_j^l \mathbf{A}_j^l \rangle = 0$, ensuring task updates of each task remain independent and preventing catastrophic forgetting. After SVD, the relationship extends to their orthonormal basis matrices $\mathbf{U}_i^l$ and $\mathbf{U}_j^l$. They span their respective projection matrices, which are $\mathbf{P}_i^l = \mathbf{U}_i^l (\mathbf{U}_i^l)^\top$, $\mathbf{P}_j^l = \mathbf{U}_j^l (\mathbf{U}_j^l)^\top$, where $(\mathbf{U}_i^l)^\top \mathbf{U}_i^l = \mathbf{I}_i^l$ and $(\mathbf{U}_j^l)^\top \mathbf{U}_j^l = \mathbf{I}_j^l$. Using the cyclic property of trace, we have

$$\text{tr}(\mathbf{P}_i^l \mathbf{P}_j^l) = \text{tr}(\mathbf{U}_i^l (\mathbf{U}_i^l)^\top \mathbf{U}_j^l (\mathbf{U}_j^l)^\top) = \text{tr}((\mathbf{U}_i^l)^\top \mathbf{U}_j^l (\mathbf{U}_j^l)^\top \mathbf{U}_i^l) \quad (13)$$

Taking the SVD of $(\mathbf{U}_i^l)^\top \mathbf{U}_j^l$, we get $(\mathbf{U}_i^l)^\top \mathbf{U}_j^l = \mathbf{X}^l \boldsymbol{\Sigma}^l (\mathbf{Y}^l)^\top$, where $\boldsymbol{\Sigma}^l = \text{diag}(\sigma_1, \dots, \sigma_p)$ and $\mathbf{X}^l, \mathbf{Y}^l$ are orthogonal matrices. Then, we have

$$(\mathbf{U}_i^l)^\top \mathbf{U}_j^l (\mathbf{U}_j^l)^\top \mathbf{U}_i^l = \mathbf{X}^l \boldsymbol{\Sigma}^l (\mathbf{Y}^l)^\top \mathbf{Y}^l (\boldsymbol{\Sigma}^l)^\top (\mathbf{X}^l)^\top = \mathbf{X}^l \boldsymbol{\Sigma}^l (\boldsymbol{\Sigma}^l)^\top (\mathbf{X}^l)^\top \quad (14)$$

Next, using the cyclic property of trace again, we have

$$\begin{aligned} \text{tr}(\mathbf{P}_i^l \mathbf{P}_j^l) &= \text{tr}(\mathbf{X}^l \boldsymbol{\Sigma}^l (\boldsymbol{\Sigma}^l)^\top (\mathbf{X}^l)^\top) \\ &= \text{tr}(\boldsymbol{\Sigma}^l (\boldsymbol{\Sigma}^l)^\top (\mathbf{X}^l)^\top \mathbf{X}^l) \\ &= \text{tr}(\boldsymbol{\Sigma}^l (\boldsymbol{\Sigma}^l)^\top) \\ &= \sum_{s=1}^p \sigma_s^2 \end{aligned} \quad (15)$$

where $\text{tr}(\mathbf{P}_i^l \mathbf{P}_j^l)$ measures how much of projection $\mathbf{P}_j^l$ lies within subspaces spanned by $\mathbf{U}_i^l$, effectively quantifying their orthogonality. Metric ϕ is defined in Eq. 12, normalizing Eq. 15 by p , which ensures dimension-independent comparisons across layers and tasks. Unlike the unbounded $\text{tr}(\mathbf{P}_i^l \mathbf{P}_j^l)$, ϕ is constrained to $[0, 1]$, where $\phi \rightarrow 0$ indicates orthogonality, validating $\phi < \epsilon$ as a criterion for subspace orthogonality. This establishes a principled foundation for our rank adaptation.

3.5 Adaptive Low-Rank Continual Learning

We now turn to providing the detailed algorithm. For task $\mathcal{T}_i$, we use $\mathbf{B}_i^l$ and $\mathbf{A}_i^l$ to represent the layer-wise low-rank matrices at layer l .

For task $\mathcal{T}_1$. In our method, we focus primarily on the subsequent tasks after the first task, as the subspace similarity metric is designed to evaluate low-rank weight subspaces between tasks. Since the first task has no previous tasks to compare, we use a fixed low rank $\mathbf{B}_1^l \mathbf{A}_1^l$ for learning.

For task $\mathcal{T}_i, i \in [2, N]$. When training task $\mathcal{T}_i$, we freeze low-rank matrices of all previous tasks $\{\mathcal{T}_m\}_{m=1}^{i-1}$. For task $\mathcal{T}_i$, CL-Rank initializes low-rank matrices $\mathbf{B}_i^{l,0} \mathbf{A}_i^{l,0}$ with a rank of 1, minimizing the impact on previously learned tasks, as done in other incremental low-rank methods (Zhao et al., 2023). At every training step t satisfying $t \equiv 0 \pmod k$, where k is updating interval, we perform SVD on $\mathbf{B}_i^{l,t} \mathbf{A}_i^{l,t}$ to obtain top $r_i^{l,t}$ left singular

vectors $\mathbf{U}_i^{l,t}$. Similarly, we compute top r_m^l left singular matrices $\{\mathbf{U}_m^l\}_{m=1}^{i-1}$ from previous tasks $\{\mathcal{T}_m\}_{m=1}^{i-1}$ low-rank matrices $\{\mathbf{B}_m^l \mathbf{A}_m^l\}_{m=1}^{i-1}$. The average subspace similarity is then obtained by calculating $\phi(\mathbf{U}_i^{l,t}, \mathbf{U}_m^l)$ between subspaces of current task $\mathcal{T}_i$ and each previous task $\mathcal{T}_m$. If average subspace similarity is below the orthogonality threshold, indicating that $\mathbf{U}_i^{l,t}$ is sufficiently orthogonal to previous ones, CL-Rank increases the rank of $\mathbf{B}_i^{l,t}$ and $\mathbf{A}_i^{l,t}$ to improve generalization without negatively impacting prior tasks. Otherwise, the rank remains unchanged to avoid interference. Complete algorithm is outlined in Algorithm 1 and Algorithm 2.

```

1: Input: Starting rank  $r_i^{l,0} = 1$ , updating interval  $k \in \mathbb{Z}_+$ , total updating steps  $T$ ,
   first step  $t = 1$ 
2: Initialize  $\mathbf{A}_i^{l,0} \in \mathbb{R}^{r_i^{l,0} \times d_2}$  as random Gaussian initialization and  $\mathbf{B}_i^{l,0} \in \mathbb{R}^{d_1 \times r_i^{l,0}}$ 
   as zero initialization
3: while  $t \leq T$  do
4:   if  $t \equiv 0 \pmod k$  then
5:     Obtain  $\mathbf{A}_i^{l,t}, \mathbf{B}_i^{l,t}$  via Algorithm 2
6:   end if
7:   Train  $\mathbf{A}_i^{l,t}, \mathbf{B}_i^{l,t}$  and obtain  $\mathbf{A}_i^{l,t+1}, \mathbf{B}_i^{l,t+1}$ 
8:    $t \leftarrow t + 1$ 
9: end while

```

Algorithm 1 Adaptive low-rank continual learning for task $\mathcal{T}_i, i \in [2, N]$

```

1: for  $m \leq i - 1$  do
2:   Compute left singular matrix of task  $\mathcal{T}_m$ :  $\mathbf{U}_m^l \leftarrow \text{SVD}(\mathbf{B}_m^l \mathbf{A}_m^l)$ 
3:   Select top  $r_m^l$  left singular vectors of  $\mathbf{U}_m^l$ 
4: end for
5: Compute top  $r_i^{l,t}$  left singular vectors from task  $\mathcal{T}_i$ :  $\mathbf{U}_i^{l,t} \leftarrow \text{SVD}(\mathbf{B}_i^{l,t} \mathbf{A}_i^{l,t})$ 
6: Compute task subspace similarity  $\phi(\mathbf{U}_i^{l,t}, \mathbf{U}_m^l)$ , where  $m = 1, \dots, i - 1$ 
7: if  $\frac{1}{i-1} \sum_{m=1}^{i-1} \phi(\mathbf{U}_i^{l,t}, \mathbf{U}_m^l) < \epsilon$  then
8:   Increase rank:  $r_i^{l,t} = r_i^{l,t} + 1$ 
9:   Initialize additional parameters:
10:   $\mathbf{A}_i^{l,t} \leftarrow [\mathbf{A}_i^{l,t}, \mathbf{A}^{l,*}], \mathbf{B}_i^{l,t} \leftarrow [\mathbf{B}_i^{l,t}, \mathbf{B}^{l,*}]$ , where  $\mathbf{A}^{l,*} \in \mathbb{R}^{1 \times d_2}$  and  $\mathbf{B}^{l,*} \in \mathbb{R}^{d_1 \times 1}$ 
   are
11:   randomly initialized with small values
12: else
13:   $\mathbf{A}_i^{l,t} \leftarrow \mathbf{A}_i^{l,t}, \mathbf{B}_i^{l,t} \leftarrow \mathbf{B}_i^{l,t}$ 
14: end if

```

Algorithm 2 Adaptive low-rank update

3.6 Orthogonal Subspace Projection

CL-Rank uses task subspace similarity metric to distinguish low-rank subspaces of task $\mathcal{T}_i$ as either (i) orthogonal region $\perp_i$ (subspaces are orthogonal to previous tasks $\{\mathcal{T}_m\}_{m=1}^{i-1}$) or (ii) non-orthogonal region $\not\perp_i$. For layerwise low-rank matrices $\mathbf{B}_i^l \mathbf{A}_i^l$ in $\perp_i$, CL-Rank safely increases their rank to enhance generalization. However, for matrices in $\not\perp_i$, rank growth is halted to avoid interference with subspaces of previously learned tasks. Thus, to mitigate the interference in $\not\perp_i$, we apply orthogonal gradient projection for low-rank matrix

update. This progressively shifts more low-rank subspaces of task $\mathcal{T}_i$ into $\perp_i$, making the rank increase and improving generalization. Leveraging the low-rank structure of LoRA, which captures critical update directions (Wang et al., 2023), we thus reduce computation and memory by aligning new subspaces orthogonally to LoRAs of prior tasks. Instead of directly ensuring orthogonality of A_i^l as in (Wang et al., 2023), we consider both B_i^l and A_i^l due to additional random initialization for B_i^l during training and enforce orthogonality of low-rank subspaces of task $\mathcal{T}_i$ and of previous tasks $\{\mathcal{T}_m\}_{m=1}^{i-1}$ during training of task $\mathcal{T}_i$ with the additional loss regularizer:

$$\lambda \sum_{l=1}^L \sum_{m=1}^{i-1} \sum_{j,k} \| [A_i^l (A_m^l)^\top]_{j,k} \|^2 + \lambda \sum_{l=1}^L \sum_{m=1}^{i-1} \sum_{j,k} \| [(B_i^l)^\top B_m^l]_{j,k} \|^2 \quad (16)$$

where $[\cdot]_{j,k}$ represents element at j -th row and k -th column, and λ is weight for each regularizer.

4 Experiments

4.1 Experimental Setup

Our experiments utilize the encoder-decoder architecture of the T5-large and T5-base models (Raffel et al., 2020) and decoder-only architecture of Llama-2-7b (Touvron et al., 2023), in line with previous work in continual learning (CL) for natural language processing. Experiments are conducted on NVIDIA A6000 GPUs (for T5) and A100 GPUs (for Llama-2-7b), leveraging the DeepSpeed repository.

4.1.1 Datasets

Standard CL benchmark We use the standard CL benchmark designed specifically for language models, comprising five text classification datasets: AG News, Amazon Reviews, Yelp Reviews, DBpedia, and Yahoo Answers, as introduced by (Zhang et al., 2015). We follow the CL setup for T5 model outlined in LFPT5 (Qin & Joty, 2021), experimenting with three different task orders within this benchmark. Large number of tasks To further assess the effectiveness of our method, we evaluate it on extended task sequences using a comprehensive CL benchmark involving 15 datasets, as described in (Razdaibiedina et al., 2023). It combines tasks from three distinct sources: five from the standard CL benchmark, four from GLUE benchmark (MNLI, QQP, RTE, SST-2), five from SuperGLUE benchmark (WiC, CB, COPA, MultiRC, BoolQ), and IMDB movie reviews dataset. For each task, we train on 1000 randomly selected samples and validate using 500 samples per class, adhering to the prior work (Razdaibiedina et al., 2023). **Math benchmarks** We utilize challenging math benchmarks, specifically GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021). GSM8K includes a collection of 8.5K grade-school math word problems and the solutions. MATH consists of problems from mathematics competitions, covering a range of difficulty levels in math. To ensure a controlled comparison and consistent training budget across benchmarks, we train on 7500 examples for each dataset. For GSM8K, this corre-

sponds to the full official training split. For MATH, we uniformly subsample 7500 examples from the official training set using a fixed random seed. For evaluation, we use the official test splits: 1000 examples from GSM8K and 5000 examples from MATH. This setup balances computational feasibility with sufficient task diversity and difficulty, enabling a fair assessment of continual learning performance on mathematical reasoning tasks.

4.1.2 Metrics

We define the testing accuracy on task $\mathcal{T}_i$ after training on task $\mathcal{T}_j$ as $a_{i,j}$. The evaluation metrics we used are: (i) **Average Accuracy (AA)**, which is computed as the mean accuracy across all tasks after completing the training on the final task: $\frac{1}{T} \sum_{i=1}^T a_{i,T}$; (ii) **Backward Transfer (BWT)** (Lopez-Paz & Ranzato, 2017; Lin et al., 2022), which is the influence that learning a task t has on the performance on a previous task $k < t$, defined as $\frac{1}{T-1} \sum_{i=1}^{T-1} (a_{i,T} - a_{i,i})$; (iii) **Forward Transfer (FWT)** (Lopez-Paz & Ranzato, 2017), which is the influence that learning a task t has on the performance on a future task $k > t$, defined as $\frac{1}{T-1} \sum_{i=2}^{T-1} (a_{i-1,i} - \bar{b}_i)$, where $\bar{b}_i$ is the vector of test accuracies for each task at random initialization; (iv) **Order-normalized Performance Disparity (OPD)** (Yoon et al., 2020): $OPD_i = \max(a_{i,T}^1, \dots, a_{i,T}^R) - \min(a_{i,T}^1, \dots, a_{i,T}^R)$, where R is the number of task orders, to further quantify order robustness, where we compute Maximum OPD as $MOPD = \max(OPD_1, \dots, OPD_T)$ and Average OPD as $AOPD = \frac{1}{T} \sum_{i=1}^T OPD_i$.

4.1.3 Baselines

We compare our method against various baselines as follows:

- SeqFT (Masson D'autume et al., 2019) trains all model parameters on a sequence of tasks (without adding any regularization or replaying samples from the previous tasks).
- SeqLoRA trains fixed-size LoRA parameters on a sequence of tasks (without adding any regularization or replaying samples from the previous tasks).
- IncLoRA that incremental learning of new LoRA parameters on a sequence of tasks (without adding any regularization or replaying samples from the previous tasks).
- Replay fine-tunes the whole model with a memory buffer and replays samples from old tasks when learning new tasks to avoid forgetting.
- EWC (Kirkpatrick et al., 2017) fine-tunes the whole model with a regularization loss that prevents updating parameters that could interfere with previously learned tasks.
- LwF (Li & Hoiem, 2017) constrains the shared representation layer to be similar to its original state before learning the new task.
- L2P (Wang et al., 2022) uses the input to dynamically select and update prompts from the prompt pool in an instance-wise fashion.
- LFPT5 (Qin & Joty, 2021) continuously trains a soft prompt that simultaneously learns to solve the tasks and generate training samples, which are subsequently used in experience replay.
- ProgPrompt (Razdaibiedina et al., 2023) adopts task-specific soft prompts for each task, training distinct models per task and using task IDs during inference.

- O-LoRA (Wang et al., 2023) incrementally trains new tasks in an orthogonal subspace while fixing LoRA matrices of previous tasks.
- SAPT-LoRA (Zhao et al., 2024) aligns learning process and selection process of LoRA via the shared module.
- PerTaskFT trains a separate model for each task individually.
- MTL trains a multi-task learning model on all tasks simultaneously, serving as the performance upper bound for the benchmark.

4.2 Main Results

Tables 1 and 3 present the performance comparisons of CL-Rank and baseline continual learning methods across two CL benchmarks using T5-large and Llama-2. In line with LFPT5, we report average results from three random runs, each with a different task order on the CL benchmark.

4.2.1 Results on Standard Continual Learning Benchmarks

Across three task orders, CL-Rank with orthogonal projection ('CL-Rank w OP' in Tables 1 and 3) achieves comparable or superior performance to previous methods. Specifically, CL-Rank with orthogonal projection shows performance improvements across all task orders compared to O-LoRA, the previous state-of-the-art without data replay. Furthermore, CL-Rank without orthogonal projection ('CL-Rank wo OP' in Tables 1 and 3) surpasses most baselines, suggesting that adaptively increasing the rank at the right time can enhance performance. SAPT-LoRA achieves state-of-the-art performance, but we need to highlight that it heavily relies on previous tasks' data samples and task labels when training new

Table 1 Comparison of testing performance on two standard CL benchmarks using the T5-large model across different task orders, where bold values indicate the best performance among methods under the data-free continual learning.

Order	Standard CL Benchmark				Large number of tasks			
	1	2	3	Avg	4	5	6	Avg
SeqFT	18.9	24.9	41.7	28.5	7.4	7.3	7.4	7.4
SeqLoRA	39.5	31.9	46.6	39.3	4.9	3.5	4.2	4.2
IncLoRA	63.4	62.2	65.1	63.6	63.0	57.9	60.4	60.5
Replay	50.3	52.0	56.6	53.0	54.5	54.3	53.5	54.1
EWC	46.3	45.3	52.1	47.9	44.9	44.0	45.4	44.8
LwF	52.7	52.9	48.4	51.3	49.7	42.8	46.9	46.5
L2P	59.0	60.5	59.9	59.8	57.7	53.6	56.6	56.0
LFPT5	66.6	71.2	76.2	71.3	69.8	67.2	69.2	68.7
CL-Rank(wo OP)	72.8	73.7	70.7	72.4	63.2	65.2	62.1	63.5
O-LoRA	74.9	75.3	75.9	75.4	70.5	65.5	70.5	68.8
CL-Rank(w OP)	77.3	77.1	76.0	76.8	69.9	69.2	71.5	70.2
SAPT-LoRA	82.9	83.8	80.9	82.5	84.7	78.9	83.2	82.3
ProgPrompt	76.1	76.0	76.3	76.1	78.7	78.8	77.8	78.4
PerTaskFT	70.0	70.0	70.0	70.0	78.1	78.1	78.1	78.1
MTL	80.0	80.0	80.0	80.0	76.3	76.3	76.3	76.3

We report the average testing accuracy after training the final task in each task order, averaged over three random runs

Table 2 Evaluation of additional continual learning metrics on the standard CL benchmark

Method	Avg Acc (%)	BWT $\uparrow$	FWT $\uparrow$
SeqLoRA	39.3	− 23.04	1.25
IncLoRA	63.6	− 22.75	1.31
O-LoRA	75.4	− 3.28	− 2.26
SAPT-LoRA	82.5	− 1.21	1.94
CL-rank	76.8	− 1.73	1.83

Table 3 Comparison of testing performance on two standard CL benchmarks using Llama-2-7b model across different task orders

Order	Standard CL Benchmark				Large number tasks				AOPD	MOPD
	1	2	3	Avg	4	5	6	Avg		
O-LoRA	76.1	76.3	76.4	76.3	72.2	70.3	71.1	71.2	5.730%	9.368%
SAPT-LoRA	83.2	81.3	84.0	82.8	82.5	80.2	83.6	82.1	5.482%	8.916%
CL-Rank(w OP)	77.5	77.6	77.3	77.5	74.4	72.8	73.1	73.4	4.953%	6.171%

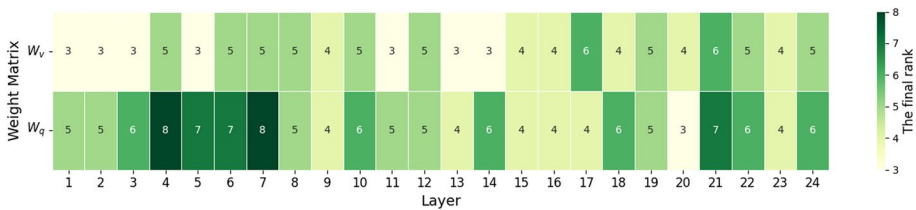
tasks, which provides it with additional supervision that is unavailable to methods under data-free assumptions. Moreover, SAPT-LoRA is less robust to task order variations compared to ours. Our approach also performs on par with MTL and surpasses PerTaskFT by a notable margin. This shows that CL-Rank with orthogonal projection effectively mitigates catastrophic forgetting and maintains adaptability. For OPD, the order-robust model has low values for both metrics (Yoon et al., 2020). CL-Rank shows the least disparity in performance to orders. While SAPT-LoRA achieves higher average accuracy, it suffers from greater disparity, likely due to pseudo-sample generation. For BWT and FWT, we report the results in Table 2. The results show that CL-Rank achieves more favorable transfer characteristics than O-LoRA, which further supports our central claim that adaptive rank allocation helps balance stability and plasticity.

4.2.2 Results on Large Number of Tasks

In benchmarks with a large number of tasks, CL-Rank with orthogonal projection outperforms O-LoRA in average performance across all three task orders. Notably, even CL-Rank without orthogonal projection surpasses IncLoRA, as our method relies solely on increasing training rank based on subspace similarity metric and updating interval, highlighting the benefit of adaptive rank adjustment. As we discussed in standard CL benchmarks, SAPT-LoRA, which heavily relies on data samples of previous tasks and task labels when training new tasks, remains less robust to task order variations. While ProgPrompt performs well in long task sequences, it has critical limitations: it is strictly tied to training tasks and depends heavily on task IDs during inference, restricting generalization for LLMs. In contrast, our method does not require task IDs during testing, making it more generalizable. However, it is worth noting that nearly all existing continual learning methods without the help of previous tasks' data still lag behind PerTaskFT and MTL, showing the challenges of continual learning with a large number of tasks (Table 3).

Table 4 Testing accuracy changes on sequential tasks from GSM8K and MATH with Llama-2-7b, where bold values indicate the best performance among methods.

Method	$\mathcal{T}_1$: GSM8K $\rightarrow \mathcal{T}_2$: MATH $\rightarrow \mathcal{T}_3$: GSM8K $\rightarrow \mathcal{T}_4$: MATH			PerTaskFT	MTL
	GSM8K	MATH	Avg		
O-LoRA	22.14 $\rightarrow$ 14.75 $\rightarrow$ 16.87 $\rightarrow$ 17.10	3.92 $\rightarrow$ 4.23 $\rightarrow$ 4.01	6.19	7.47	9.32
CL-Rank	2.14 $\rightarrow$ 15.31 $\rightarrow$ 18.54 $\rightarrow$ 19.22	4.34 $\rightarrow$ 4.98 $\rightarrow$ 5.21	7.55	7.47	9.32
Method	$\mathcal{T}_1$: MATH $\rightarrow \mathcal{T}_2$: GSM8K $\rightarrow \mathcal{T}_3$: MATH $\rightarrow \mathcal{T}_4$: GSM8K			PerTaskFT	MTL
	MATH	GSM8K	Avg		
O-LoRA	4.52 $\rightarrow$ 1.44 $\rightarrow$ 2.34 $\rightarrow$ 2.32	18.45 $\rightarrow$ 17.39 $\rightarrow$ 15.82	4.57	7.47	9.32
CL-Rank	4.52 $\rightarrow$ 3.94 $\rightarrow$ 4.23 $\rightarrow$ 4.18	19.32 $\rightarrow$ 17.56 $\rightarrow$ 16.85	6.29	7.47	9.32

**Fig. 2** Comparison of the sum of the resulting ranks in the different modules of decoder layers in CL-Rank training last three tasks of order 3 on the standard CL benchmark using T5-large model. Encoder layers are provided in Appendix. 7a. Here, the x-axis is the layer index and the y-axis represents different modules (types) of low-rank weight matrices

4.2.3 Forgetting Performance on Math Benchmarks

We evaluate our method on two challenging math datasets, GSM8K and MATH, each evenly split into two subsets, forming four tasks. To assess the impact on forgetting, we experiment with two distinct task orders: training first on GSM8K followed by MATH, and vice versa. This setup allows us to evaluate how task order influences the model's forgetting performance, and our average testing accuracy is the exact prediction accuracy of the final answer for each task, aligned with other works (Magister et al., 2023; Chung et al. 2024). Table 4 presents the testing accuracy trends for each dataset after successive tasks training. Across both task orders, CL-Rank with orthogonal projection consistently outperforms O-LoRA in final average testing accuracy and indicates less forgetting on the first task and better generalization on the second task. This suggests that CL-Rank is more effective at mitigating forgetting, especially when switching between tasks in math benchmarks. Importantly, we do not claim that CL-Rank enables strong standalone mathematical reasoning in an absolute sense, since mathematical reasoning benchmarks are widely recognized as difficult for parameter-efficient fine-tuning, particularly in continual learning scenarios with limited per-task data and frozen backbones. Instead, our goal is to evaluate continual learning behavior under an intentionally challenging and resource-constrained setting. Within this context, our claim is that CL-Rank consistently improves continual learning dynamics relative to prior LoRA-based CL methods. Specifically, CL-Rank exhibits reduced forgetting on earlier math tasks and improved adaptation to later tasks compared to O-LoRA, despite operating under the same data and parameter constraints. These improvements are reflected in task-level accuracy changes along the sequence, rather than in raw final accuracy alone.

Table 5 Comparison of CL-Rank and O-LoRA in performance of computation cost and storage

	LoRA storage	Virtual mem	Storage complexity	Rank
CL-Rank	78.8 MB	78.6GB	$O(TN(m+n)r_1)$	Adaptive
O-LoRA	121.5 MB	76.4GB	$O(TN(m+n)r_1)$	Fixed

Table 6 Scalability analysis under different task sequence lengths

Method	# Tasks	Runtime	Tasks	Runtime
O-LoRA	4	01:19:35	15	02:46:39
CL-Rank	4	01:58:39	15	04:27:45

4.3 Discussions

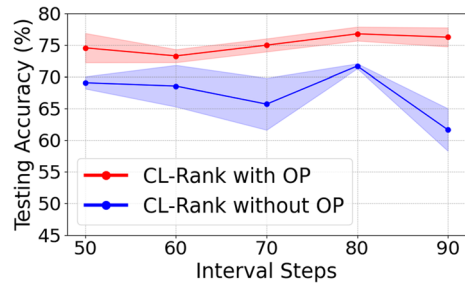
Final rank distribution across different layers Fig. 2 shows the sum of the resulting rank of low-rank matrices trained on the last three tasks of Order 3 in standard CL benchmark since the rank for the first task in our setting is not affected by other tasks. We find that the rank distribution in v modules varies more than in q modules of encoder layers, while in decoder layers, q and v modules exhibit different patterns of variability. Meanwhile, the rank distribution in encoder layers is slightly more consistent compared to decoder layers. These suggest that different layers within the model fulfill distinct roles, which our method effectively leverages to achieve better overall performance.

Evaluation on computation cost and storage We use training results from standard CL benchmark and large number of tasks with Llama-2-7b model on an NVIDIA A100 GPU, to provide (1) actual LoRA storage after training 15 tasks in large number of tasks, (2) virtual memory usage, and (3) theoretical computational complexity in Table 5. We first define T as the number of prior tasks and N as the number of layers. For computational complexity, both methods grow with sequence length during training, but this is a common challenge in LoRA-based CL methods that interact with previously learned LoRAs. Our method offers a compensatory benefit: it adaptively controls and often reduces the effective rank r_1 of new LoRAs, making our LoRA storage usage less than O-LoRA. SVD operations in our method make virtual memory usage slightly higher than O-LoRA, but these costs are controlled by the updating interval since we do not compute SVD at each step.

Importantly, the SVD operations in CL-Rank are performed on low-rank LoRA matrices rather than full model parameters. In practice, these matrices are small (with ranks typically far below the hidden dimension), and SVD is applied only at task boundaries rather than at every training step. As a result, the dominant training cost remains LoRA fine-tuning, while the SVD overhead constitutes a relatively small fraction of total runtime. Although CL-Rank compares the current task subspace against previously learned subspaces, the effective interaction cost is mitigated by the fact that the stored subspaces are compact orthonormal bases whose dimensionality equals the learned rank. Moreover, rank growth is periodic, so the cumulative subspace dimensionality grows slowly during fine-tuning, preventing quadratic blow-up in practice. From a continual learning perspective, the overhead introduced by CL-Rank directly enables adaptive rank allocation and reduced interference across tasks. We explore and utilize the mystery of the role of rank, which is underexplored in continual learning. Our experiments show that this additional cost yields substantial improvements in stability–plasticity trade-offs compared to fixed-rank baselines, indicating that the overhead is justified by improved continual learning behavior. To assess the computational scalability

Table 7 Comparison of CL-Rank performance with different ϵ across three task orders on standard CL benchmark using T5-large model

CL-Rank w OP	0.001	0.005	0.01	0.05	0.1	CL-Rank wo OP	0.1	0.5
Order 1	77.3	78.3	75.0	76.8	75.7	Order 1	70.0	72.8
Order 2	77.1	77.3	76.3	75.4	76.7	Order 2	70.8	73.7
Order 3	77.1	76.7	76.2	74.3	73.6	Order 3	69.9	70.7

Fig. 3 CL-Rank w/wo OP with different updating intervals across three task orders on the standard CL benchmark using the T5-large model

of CL-Rank, we report the wall-clock runtime on continual learning benchmarks using the Llama-2-7B model under task sequences of different lengths. The results are summarized in Table 6. The results show that although CL-Rank requires additional SVD computations, the overhead grows moderately with the number of tasks and remains practical in the evaluated settings. Furthermore, the adaptive-rank mechanism reduces the effective LoRA storage compared with fixed-rank approaches, partially compensating for the additional computational cost.

Impact of subspace similarity threshold ϵ We evaluate the performance for different values of ϵ (0.001, 0.005, 0.01, 0.05, 0.1, 0.5), as shown in Table 7. The results indicate that accuracy remains highly stable at $\epsilon = 0.001$ for CL-Rank with orthogonal projection and at $\epsilon = 0.1$ for CL-Rank without orthogonal projection. There is a slight downtrend between 0.001 and 0.1 in CL-Rank with orthogonal projection, which occurs because orthogonal projection ensures that more subspaces are orthogonal to the previously learned subspaces, and a larger threshold would make negatively affected subspaces increase in rank, thus worsening the results. In CL-Rank without orthogonal projection, accuracy at 0.5 is slightly better than 0.1, suggesting that more subspaces are treated as orthogonal to generalize better via increasing rank. Overall, the threshold ϵ provides a concrete criterion for determining when the inter-task subspaces can be viewed as approximately orthogonal, based on the subspace similarity analysis in Sects. 3.2–3.4, where the dominant term in the forgetting upper bound depends on the overlap between low-rank update subspaces. The criterion $\phi(U_i^l, U_j^l) < \epsilon$ thus serves as an interpretable rule for deciding when rank expansion is not to interfere with previously learned tasks, rather than an arbitrary heuristic. As shown in Table 7, performance varies smoothly as ϵ changes, indicating that there is an effective range for ϵ values rather than a single finely tuned value. The observed trends are consistent with the expected stability–plasticity trade-off.

Impact of updating interval k The updating interval k determines the frequency of measuring the subspace similarity threshold ϵ . To analyze the effect of updating interval k , we vary k across $\{50, 60, 70, 80, 90\}$ while keeping other hyperparameters the same. Figure 3 shows that the accuracy of CL-Rank with orthogonal projection remains stable across dif-

Table 8 Comparison of different rank patterns across methods using T5-large on standard CL benchmark, where bold values indicate the best performance among methods.

'r(avg)' denotes the average per-task per-layer rank (rounded to integer), averaging the sum of incremental LoRA ranks across all layers for the last three tasks

Order	1	2	3	Avg	r(Avg)
CL-Rank w OP	77.3	77.1	76.0	76.8	3
CL-Rank wo OP	72.8	73.7	70.7	72.4	4
IncLoRA ($r = 2$)	44.5	48.5	50.7	47.9	2
IncLoRA ($r = 4$)	50.4	44.0	56.7	50.4	4
IncLoRA ($r = 8$)	63.4	62.2	65.1	63.6	8
IncLoRA ($r = 16$)	62.5	62.4	67.5	64.1	16
O-LoRA ($r = 2$)	73.5	73.2	74.4	73.7	2
O-LoRA ($r = 4$)	75.7	75.6	75.4	75.6	4
O-LoRA ($r = 8$)	74.9	75.3	75.9	75.4	8
O-LoRA ($r = 16$)	75.2	74.9	76.9	75.7	16

Table 9 Comparison of different models' performances across three task orders on the standard CL benchmark, where bold values indicate the best performance among methods.

T5-base	Order			Avg	MTL
	1	2	3		
O-LoRA	72.9	72.3	72.6	72.6	78.3
CL-Rank	74.0	72.7	72.1	72.9	78.3
T5-large	1	2	3	Avg	MTL
O-LoRA	74.9	75.3	75.9	75.4	80.0
CL-Rank	77.3	77.1	76.0	76.8	80.0

ferent values of k , suggesting that CL-Rank does not depend on a single specifically tuned value. In contrast, CL-Rank without orthogonal projection exhibits relatively stable performance between $k=50$ and $k=80$ but experiences a sharp decline at $k=90$, since in this case, larger updating intervals cannot grasp rapid changes in subspaces and would miss critical changing points. Importantly, the updating interval k does not change the decision rule for rank expansion, but only the temporal granularity at which the rule is applied.

Different fixed ranks performance Table 8 presents average testing accuracy in different fixed rank settings compared with CL-Rank across different orders. CL-Rank with orthogonal projection, using rank 8 for the first task (as O-LoRA), consistently outperforms all fixed rank baselines. Moreover, CL-Rank without orthogonal projection achieves performance comparable to O-LoRA and outperforms IncLoRA, which suggests that adaptively increasing rank at the right time can enhance CL performance. Both IncLoRA and O-LoRA show improvements in average accuracy as rank increases, but the difference is that O-LoRA exhibits minimal difference between $r=2$ and $r=6$, while IncLoRA shows a noticeable gap between $r=2$ and $r=8$ with diminishing gains beyond $r=8$. Importantly, CL-Rank with orthogonal projection achieves superior performance while maintaining an average rank per task between $r=2$ and $r=4$, making it more memory-efficient compared to higher fixed-rank settings.

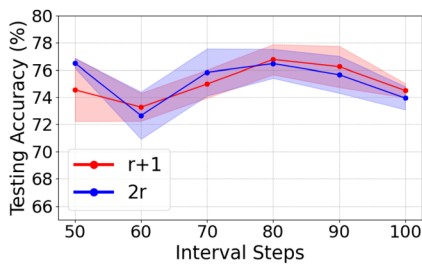
Different pre-trained models influence performances We investigate the impact of model scale on performance by comparing T5-base and T5-large models on the standard CL benchmark. We evaluate both CL-Rank with orthogonal projection and O-LoRA across three task orders. The results, shown in Table 9, present the performance differences between the

Table 10 Comparison of CL-Rank performance across three task orders of the standard CL benchmark with varying fixed ranks for the first task, using T5-large model

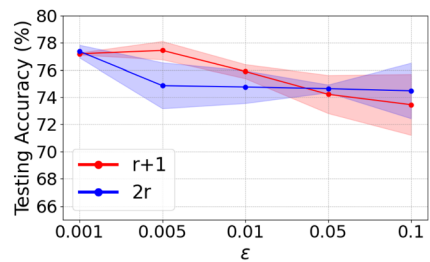
Order	1	2	3	Avg
$r_1 = 2$	76.7	76.3	76.1	76.4
$r_1 = 4$	76.9	76.4	76.1	76.5
$r_1 = 8$	77.3	77.1	76.0	76.8
$r_1 = 16$	77.2	76.7	76.3	76.7

Table 11 Effect of different task sample sizes on continual learning performance

Method	1000	2000	Full dataset
O-LoRA	75.4	76.7	77.2
CL-Rank	76.8	77.0	77.4



(a) Different updating intervals in two different updating rules.

(b) Different ϵ values in two different updating rules.**Fig. 4** CL-Rank performance across three task orders on standard CL benchmark using T5-large model: **a** updating intervals under different rank-increasing rules, **b** subspace similarity thresholds ϵ under different rank-increasing rules

two model sizes and the methods employed. For T5-base model, CL-Rank with orthogonal projection consistently outperforms O-LoRA. While for T5-large model, CL-Rank significantly surpasses O-LoRA's outcomes. Moreover, CL-Rank shows exceptional consistency across all task orders in T5-large model, highlighting its robustness and effectiveness when the model size is scaled up.

Impact of the fixed rank of the first task Table 10 shows the performance of CL-Rank with different fixed ranks of the first task. We use CL-Rank with orthogonal projection to evaluate the standard CL benchmark. It suggests that increasing the fixed rank for the first task might slightly improve the final average accuracy, but differences between different ranks are relatively modest, where there is not a significant gap between $r_1 = 2$ and $r_1 = 16$. It indicates that CL-Rank does not heavily depend on the specific rank of the first task but maintains robust performance across a variety of initial rank settings.

Impact of different task sample sizes To evaluate whether the effectiveness of CL-Rank extends beyond the limited-data setting, we further compare CL-Rank and O-LoRA using different numbers of training samples per task, including 1,000 samples, 2,000 samples, and

the full training dataset. The results are summarized in Table 11. As the number of training samples increases, both methods achieve improved performance. However, CL-Rank consistently outperforms O-LoRA across all training sizes. These results show that the advantages of CL-Rank are not limited to small-data continual learning scenarios, but remain effective as substantially larger task-specific training sets become available.

Difference between different increasing rules ($2r$ v.s. $r + 1$) To evaluate the effectiveness of our updating rule $\text{update}(r) = r + 1$, we compare it with another common updating rule $\text{update}(r) = 2r$, which is mentioned in (Cosson et al., 2022) that $r + 1$ can be advantageously replaced with $2r$ in some cases. Figure 4a shows that the changing patterns of the two updating rules almost overlapped across different updating intervals. Figure 4b illustrates the performance of these two different updating rules across subspace similarity thresholds ϵ . When varying subspace similarity thresholds within the range from 0.001 to 0.01, the updating rule $r + 1$ is more stable, whereas $2r$ drops sharply at 0.005. As the threshold increases, the updating rule $r + 1$ experiences a performance drop and becomes variable, but $2r$ remains stable at lower accuracy. These results suggest that $r + 1$ is more effective within a certain threshold range, which is why we chose it as our updating rule.

5 Conclusion

We propose a parameter-efficient continual learning method, CL-Rank, which adaptively increases the layerwise rank of incremental LoRAs for new tasks based on the inter-task low-rank subspace similarity metric. CL-Rank not only accounts for the low-rank relationships between incremental LoRAs of tasks but also adapts to the unique low-rank dynamics across different model layers. This approach effectively mitigates forgetting of previous tasks while enhancing generalization on unseen data in a memory-efficient way. We perform extensive experiments on both natural language processing benchmarks and challenging math reasoning tasks via different structured models, indicating that CL-Rank captures rank patterns effectively in continual learning and shows comparable or superior performance over existing methods. However, CL-Rank is tailored to LoRA-based continual fine-tuning scenarios and may not directly generalize to large-scale pretraining settings.

Appendix A Implementation Details

We conducted experiments on T5 and Llama-2-7b models using high-performance GPU servers with DeepSpeed for efficient training. For T5 models, we used four NVIDIA A6000 GPUs with a learning rate of $1e-3$, a total batch size of 32 (8 per GPU), a dropout rate of 0.1, and no weight decay. For Llama-2-7b, we used four NVIDIA A100 GPUs with a learning rate of $1e-4$, a total batch size of 4 (1 per GPU), the same dropout rate, and no weight decay. The experimental setup remained consistent across all task sequences and orders.

Datasets

Continual Learning Benchmarks

Tables 12, 13 and 14 provide detailed information on the 15 datasets used in our continual learning (CL) experiments, along with the evaluation metrics employed. The selected datasets include those from well-established benchmarks: the standard CL benchmark (Zhang et al., 2015), GLUE (Wang et al., 2018), and SuperGLUE benchmarks (Wang et al., 2019), as well as the IMDB movie reviews dataset.

Table 12 The details of 15 datasets used in our CL experiments

Dataset name	Category	Task	Domain	Metric
1. Yelp	CL Benchmark	Sentiment Analysis	Yelp Reviews	Accuracy
2. Amazon	CL Benchmark	Sentiment Analysis	Amazon Reviews	Accuracy
3. DBpedia	CL Benchmark	Topic Classification	Wikipedia	Accuracy
4. Yahoo	CL Benchmark	Topic Classification	Yahoo Q&A	Accuracy
5. AG News	CL Benchmark	Topic Classification	News	Accuracy
6. MNLI	GLUE	NLI	Various	Accuracy
7. QQP	GLUE	Paragraph Detection	QUora	Accuracy
8. RTE	GLUE	NLI	News, Wikipedia	Accuracy
9. SST-2	GLUE	Sentiment Analysis	Movie Reviews	Accuracy
10. WiC	SuperGLUE	Word Sense Disambiguation	Lexical Databases	Accuracy
11. CB	SuperGLUE	NLI	Various	Accuracy
12. COPA	SuperGLUE	QA	Blogs, Encyclopedia	Accuracy
13. BoolQA	SuperGLUE	Boolean QA	Wikipedia	Accuracy
14. MultiRC	SuperGLUE	QA	Various	Accuracy
15. IMDB	SuperGLUE	Sentiment Analysis	Movie Reviews	Accuracy

NLI denotes natural language inference, QA denotes questions and answers task. The first five tasks correspond to the standard CL benchmark, all other tasks are used in long-sequence experiments

Table 13 Six different task sequence orders utilized in continual learning experiments

Order	Model	Task sequence
1	T5-large, T5-base, Llama-2-7b	dbpedia → amazon → yahoo → ag
2	T5-large, T5-base, Llama-2-7b	dbpedia → amazon → ag → yahoo
3	T5-large, T5-base, Llama-2-7b	yahoo → amazon → ag → dbpedia
4	T5-large, Llama-2-7b	mnli → cb → wic → copa → qqp → boolqa → rte → imdb → yelp → amazon → sst-2 → dbpedia → ag → multirc → yahoo
5	T5-large, Llama-2-7b	multirc → boolqa → wic → mnli → cb → copa → qqp → rte → imdb → sst-2 → dbpedia → ag → yelp → amazon → yahoo
6	T5-large, Llama-2-7b	yelp → amazon → mnli → cb → copa → qqp → rte → imdb → sst-2 → dbpedia → ag → yahoo → multirc → boolqa → wic

Orders 1-3 follow the standard continual learning benchmark as established by previous research, focusing on a more traditional task sequence. Orders 4-6 customized for long-sequence experimentation, encompass 15 tasks each and are structured according to the methodologies outlined in (Razdaibiedina et al., 2023)

Table 14 Instructions for different tasks

Task	Prompts
NLI	What is the logical relationship between the “sentence 1” and the “sentence 2”? Choose one from the option
QQP	Whether the “first sentence” and the “second sentence” have the same meaning? Choose one from the option
SC	What is the sentiment of the following paragraph? Choose one from the option
TC	What is the topic of the following paragraph? Choose one from the option
BoolQA	According to the following passage, is the question true or false? Choose one from the option
MultiRC	According to the following passage, is the question true or false? Choose one from the option
WiC	Given a word and two sentences, whether the word is used with the same sense in both sentences? Choose one from the option

Math Benchmarks

Tables 15 and 16 show the data structure of both GSM8K and MATH, and Table 17 provides the task information and evaluation metric for math datasets.

Table 15 Data structure of GSM8K dataset

Data field	Data content
Question	Natalia sold clips to 48 of her friends in April, and then she sold half as many Clips in May. How many clips did Natalia sell altogether in April and May?
Answer	Natalia sold $48/2 = << 48/2 = 24 >> 24$ clips in May Natalia sold $48 + 24 = << 48 + 24 = 72 >> 72$ clips altogether in April and May #####72

Table 16 Data structure of MATH dataset

Data field	Data content
Problem	A board game spinner is divided into three parts labeled \$A\$, \$B\$ and \$C\$ The probability of the spinner landing on \$A\$ is $\frac{1}{3}$ and the Probability of the spinner landing on \$B\$ is $\frac{5}{12}$. What is the Probability of the spinner landing on \$C\$? Express your answer as a common Fraction
Level	Level 1
Type	Counting & Probability
Solution	The spinner is guaranteed to land on exactly one of the three regions, so we Know that the sum of the probabilities of it landing in each region will be 1. If We let the probability of it landing in region \$C\$ be \$x\$, we then have the Equation $1 = \frac{5}{12} + \frac{1}{3} + x$ From which we have $x = \boxed{\frac{1}{4}}$

Table 17 The details of GSM8K and MATH datasets used in our CL experiments

Dataset name	Category	Task	Metric
1. GSM8K	Math Benchmark	Math Reasoning	Exact Prediction Accuracy
2. MATH	Math Benchmark	Math Reasoning	Exact Prediction Accuracy

For the matric, we use the exact prediction accuracy for evaluating these two datasets, which is the correction rate of the final answer

Forgetting Performance on Math Benchmarks

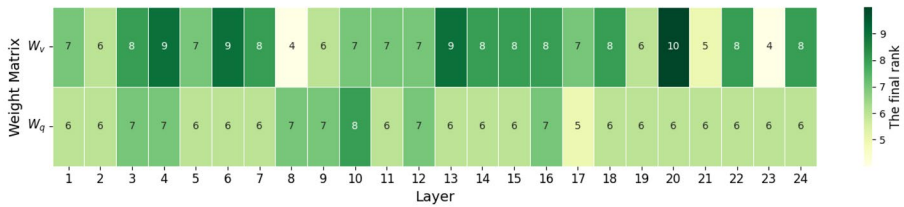
We evaluate our method on two challenging math datasets, GSM8K and MATH, each evenly split into two subsets, forming four tasks. To assess the impact on forgetting, we experiment with two distinct task orders: training first on GSM8K followed by MATH, and vice versa. This setup allows us to evaluate how task order influences the model's forgetting performance, and our average testing accuracy is the exact prediction accuracy of the final answer for each task, aligned with other works (Magister et al., 2023; Chung et al. 2024). Table 18 presents the testing accuracy trends for each dataset after successive tasks training. Across both task orders, CL-Rank with orthogonal projection consistently outperforms O-LoRA in final average testing accuracy and demonstrates less forgetting on the first task and better generalization on the second task. This suggests that CL-Rank is more effective at mitigating forgetting, especially when switching between tasks in math benchmarks. Additionally, we note that fully fine-tuning MATH using T5-large model achieves approximately 3.0%, while fully fine-tuning GSM8K using T5-large model approaches 4.2%, as also reported in the work (Magister et al., 2023).

Table 18 Comparison of testing accuracy changes on sequential tasks from GSM8K and MATH with T5-large model

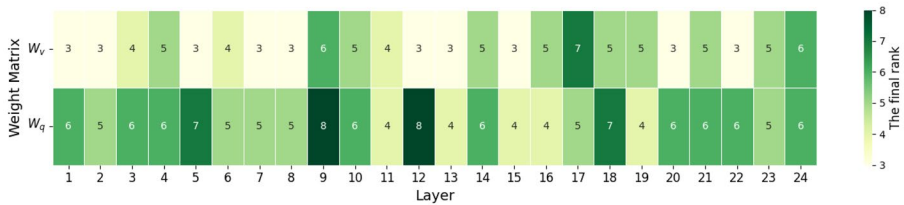
$\mathcal{T}_1$: GSM8K $\rightarrow \mathcal{T}_2$: MATH $\rightarrow \mathcal{T}_3$: GSM8K $\rightarrow \mathcal{T}_4$: MATH					
Method	GSM8K	MATH	Avg	PerTaskFT	MTL
O-LoRA	1.87 $\rightarrow$ 0.01 $\rightarrow$ 0.22 $\rightarrow$ 0.23	1.18 $\rightarrow$ 1.34 $\rightarrow$ 1.6	1.32	3.63	3.88
CL-Rank	1.87 $\rightarrow$ 0.08 $\rightarrow$ 0.76 $\rightarrow$ 1.1	1.58 $\rightarrow$ 2.32 $\rightarrow$ 2.38	2.23	3.63	3.88
$\mathcal{T}_1$: MATH $\rightarrow \mathcal{T}_2$: GSM8K $\rightarrow \mathcal{T}_3$: MATH $\rightarrow \mathcal{T}_4$: GSM8K					
Method	MATH	GSM8K	Avg	PerTaskFT	MTL
O-LoRA	2.62 $\rightarrow$ 1.44 $\rightarrow$ 2.34 $\rightarrow$ 2.32	0.30 $\rightarrow$ 0.15 $\rightarrow$ 0.14	1.87	3.63	3.88
CL-Rank	2.62 $\rightarrow$ 2.54 $\rightarrow$ 2.62 $\rightarrow$ 2.48	1.21 $\rightarrow$ 1.14 $\rightarrow$ 0.38	2.04	3.63	3.88

Resulting Rank Distributions Across Different Layers

Figures 5, 6 and 7 show the sum of the final learned ranks of the last three tasks in order 1, 2, and 3 for the standard CL benchmark using T5-large model. We find that the rank distribution in the v modules varies more than in the q modules of encoder layers, while in the decoder layers, the q and v modules exhibit different patterns of variability. Meanwhile, the rank distribution in the encoder layers is slightly more consistent compared to the decoder layers. These findings suggest that different layers within the model fulfill distinct roles, which our method effectively leverages to achieve better overall performance.

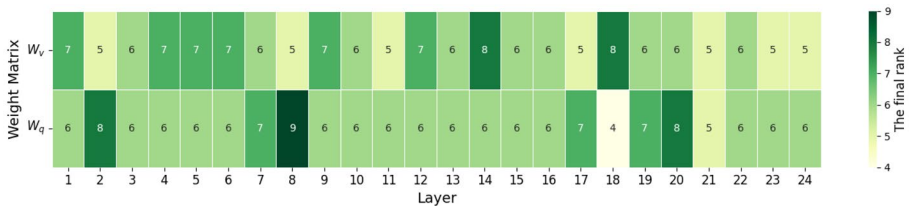


(a) Encoder Layer Final Rank

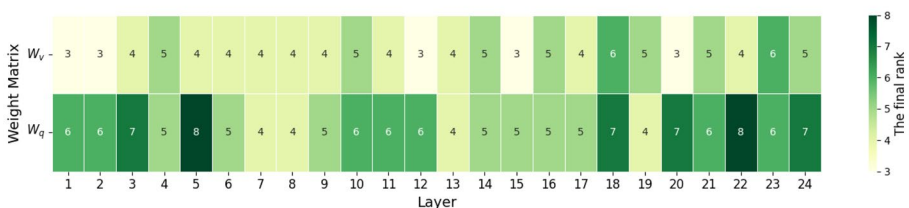


(b) Decoder Layer Final Rank

Fig. 5 Comparison of the sum of the resulting ranks in the different modules of encoder (a) and decoder (b) Layers in CL-Rank training last three tasks of order 1 on standard CL benchmark. Here the x-axis is the layer index and the y-axis represents different modules (types) of low-rank weight matrices

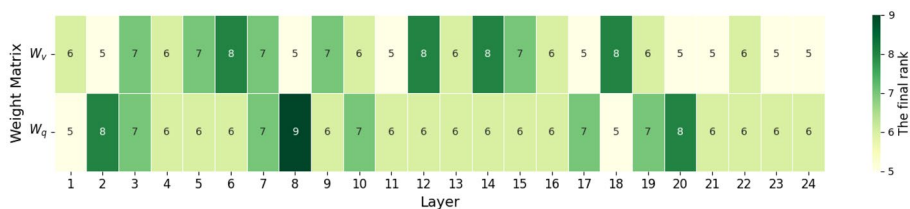


(a) Encoder Layer Final Rank

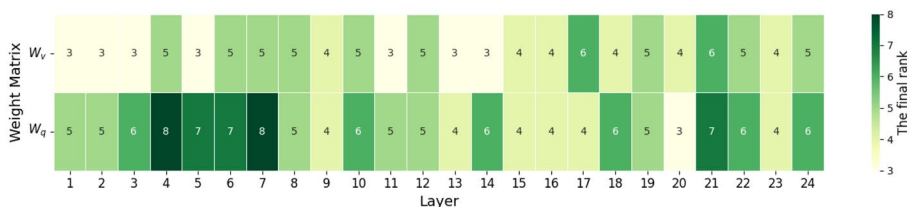


(b) Decoder Layer Final Rank

Fig. 6 Comparison of the sum of the resulting ranks in the different modules of encoder (a) and decoder (b) Layers in CL-Rank training last three tasks of order 2 on standard CL benchmark. Here the x-axis is the layer index and the y-axis represents different modules (types) of low-rank weight matrices



(a) Encoder Layer Final Rank



(b) Decoder Layer Final Rank

Fig. 7 Comparison of the sum of the resulting ranks in the different modules of encoder (a) and decoder (b) layers in CL-Rank training last three tasks of order 3 on standard CL benchmark. Here, the x-axis is the layer index and the y-axis represents different modules (types) of low-rank weight matrices

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1007/s10994-026-07145-4>.

Acknowledgements This work is partially supported by NSF EFMA Award # 2318101.

Author Contributions All authors contributed to proposing ideas for this work, and contributed to validating the proposed method. All authors contributed to writing and polishing the final manuscript. Experiments were designed and conducted by Fuli Qiao under the supervision of Mehrdad Mahdavi.

Funding This work is partially supported by NSF EFMA Award # 2318101.

Data Availability All data in the experiments are public data, which can be found at: https://huggingface.co/datasets/aps/super_glue and <https://huggingface.co/datasets/nyu-ml/glue>.

Code Availability Code is provided in the [link](#).

Declarations

Conflict of interest The authors declare no conflict of interest.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., & Anadkat, S. (2023). Gpt-4 technical report [arXiv:2303.08774](https://arxiv.org/abs/2303.08774) arXiv preprint.
- Biderman, D., Ortiz, J. G., Portes, J., Paul, M., Greengard, P., Jennings, C., King, D., Havens, S., Chiley, V., & Frankle, J. (2024). Lora learns less and forgets less [arXiv:2405.09673](https://arxiv.org/abs/2405.09673) arXiv preprint.
- Carpenter, G. A., & Grossberg, S. (1987). A massively parallel architecture for a self-organizing neural pattern recognition machine. *Computer Vision, Graphics, and Image Processing*, 37(1), 54–115.

- Chung, H. W., Hou, L., Longpre, S., Zoph, B., Tay, Y., Fedus, W., Li, Y., Wang, X., Dehghani, M., Brahma, S., et al. (2024). Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70), 1–53.
- Cosson, R., Jadbabaie, A., Makur, A., Reisizadeh, A., & Shah, D. (2022). Gradient descent for low-rank functions [arXiv:2206.08257](#) arXiv preprint.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., & Nakano, R. (2021). Training verifiers to solve math word problems [arXiv:2110.14168](#) arXiv preprint.
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. (2023). Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240), 1–113.
- Ding, N., Lv, X., Wang, Q., Chen, Y., Zhou, B., Liu, Z., & Sun, M. (2023). Sparse low-rank adaptation of pre-trained language models. The 2023 Conference on Empirical Methods in Natural Language Processing
- Dong, Y. (2024). Adarank: Disagreement based module rank prediction for low-rank adaptation [arXiv:2408.09015](#) arXiv preprint.
- Evron, I., Moroshko, E., Ward, R., Srebro, N., & Soudry, D. (2022). How catastrophic can catastrophic forgetting be in linear regression? *Conference on Learning Theory* (pp. 4028–4079) PMLR.
- Farajtabar, M., Azizan, N., Mott, A., & Li, A. (2020). Orthogonal gradient descent for continual learning. *International Conference on Artificial Intelligence and Statistics* (pp. 3762–3773) PMLR.
- Grossberg, S. (2013). Adaptive resonance theory: How a brain learns to consciously attend, learn, and recognize a changing world. *Neural Networks*, 37, 1–47. Twenty-fifth Anniversary Commemorative Issue.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., & Steinhardt, J. (2021). Measuring mathematical problem solving with the math dataset [arXiv:2103.03874](#) arXiv preprint.
- Hao, Y., Cao, Y., & Mou, L. (2024). Flora: Low-rank adapters are secretly gradient compressors. Forty-first International Conference on Machine Learning
- Han, X., Dai, Y., Gao, T., Lin, Y., Liu, Z., Li, P., Sun, M., & Zhou, J. (2020). Continual relation learning via episodic memory activation and reconsolidation. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 6429–6440)
- Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., Attariyan, M., & Gelly, S. (2019). Parameter-efficient transfer learning for nlp. *International Conference on Machine Learning* (pp. 2790–2799) PMLR.
- Hamm, J., & Lee, D. D. (2008). Grassmann discriminant analysis: a unifying view on subspace-based learning. *Proceedings of the 25th International Conference on Machine Learning* (pp. 376–383)
- Hyder, R., Shao, K., Hou, B., Markopoulos, P., Prater-Bennette, A., & Asif, S. (2022). Continual Learning via Low-Rank Network Updates
- Hu, E.J., Shen, Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: LoRA: Low-rank adaptation of large language models. In: International Conference on Learning Representations (2022)
- He, J., Zhou, C., Ma, X., Berg-Kirkpatrick, T., & Neubig, G. (2021). Towards a unified view of parameter-efficient transfer learning [arXiv:2110.04366](#) arXiv preprint.
- Jia, Z., Wang, A., Qu, X., Yang, X., Wang, J.: Hierarchical-task-aware multi-modal mixture of incremental LoRA experts for embodied continual learning. In: Che, W., Nabende, J., Shutova, E., Pilehvar, M.T. (eds.) Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 28415–28427. Association for Computational Linguistics, Vienna, Austria (2025)
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521–3526.
- Lester, B., Al-Rfou, R., & Constant, N. (2021). The power of scale for parameter-efficient prompt tuning [arXiv:2104.08691](#) arXiv preprint.
- Li, Z., & Hoiem, D. (2017). Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12), 2935–2947.
- Lopez-Paz, D., & Ranzato, M. (2017). *Advances in neural information processing systems* (p. 30) Gradient episodic memory for continual learning.
- Li, H., Wu, J., & Braverman, V. (2023). Fixed design analysis of regularization-based continual learning. *Conference on Lifelong Learning Agents* (pp. 513–533) PMLR.
- Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C.F., Cheng, K.-T., Chen, M.-H.: Dora: Weight-decomposed low-rank adaptation. [arXiv:2402.09353](#) (2024)
- Lin, S., Yang, L., Fan, D., & Zhang, J. (2022). Beyond not-forgetting: Continual learning with backward knowledge transfer. *Advances in Neural Information Processing Systems*, 35, 16165–16177.
- Lu, Y., Zhang, S., Cheng, D., Xing, Y., Wang, N., WANG, P., Zhang, Y.: Visual prompt tuning in null space for continual learning. In: The Thirty-eighth Annual Conference on Neural Information Processing Systems (2024)

- Li, X., Zhou, Y., Wu, T., Socher, R., & Xiong, C. (2019). Learn to grow: A continual structure learning framework for overcoming catastrophic forgetting. *International Conference on Machine Learning* (pp. 3925–3934) PMLR.
- McCloskey, M., Cohen, N.J.: Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, vol. 24, pp. 109–165. Academic Press (1989)
- Madotto, A., Lin, Z., Zhou, Z., Moon, S., Crook, P., Liu, B., Yu, Z., Cho, E., & Wang, Z. (2020). Continual learning in task-oriented dialogue systems [arXiv:2012.15504](#) arXiv preprint.
- Magister, L.C., Mallinson, J., Adamek, J., Malmi, E., Severyn, A.: Teaching small language models to reason. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 1773–1781 (2023)
- Masson D’Autume, C., Ruder, S., Kong, L., Yogatama, D.: Episodic memory in lifelong language learning. *Advances in Neural Information Processing Systems* **32** (2019)
- Qin, C., & Joty, S. (2021). Lfpt5: A unified framework for lifelong few-shot language learning based on prompt tuning of t5 [arXiv:2110.07298](#) arXiv preprint.
- Rolnick, D., Ahuja, A., Schwarz, J., Lillicrap, T., & Wayne, G. (2019). Experience replay for continual learning. *Advances in Neural Information Processing Systems*. <https://doi.org/10.48550/arXiv.1811.11682>
- Ratcliff, R. (1990). Connectionist models of recognition memory: Constraints imposed by learning and forgetting functions. *Psychological Review*, 97(2), 285.
- Razdaibiedina, A., Mao, Y., Hou, R., Khabsa, M., Lewis, M., & Almahairi, A. (2023). Progressive prompts: Continual learning for language models. In: *The Eleventh International Conference on Learning Representations*
- Rusu, A. A., Rabinowitz, N. C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Pascanu, R., & Hadsell, R. (2016). Progressive neural networks [arXiv:1606.04671](#) arXiv preprint.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 1–67.
- Rao, D., Visin, F., Rusu, A., Pascanu, R., Teh, Y. W., & Hadsell, R. (2019). Continual unsupervised representation learning. *Advances in Neural Information Processing Systems*. <https://doi.org/10.48550/arXiv.1910.14481>
- Smith, J. S., Hsu, Y.-C., Zhang, L., Hua, T., Kira, Z., Shen, Y., & Jin, H. (2023). Continual diffusion: Continual customization of text-to-image diffusion with c-lora [arXiv:2304.06027](#) arXiv preprint.
- Song, C., Han, X., Zeng, Z., Li, K., Chen, C., Liu, Z., Sun, M., Yang, T.: Conpet: Continual parameter-efficient tuning for large language models. arXiv preprint [arXiv:2309.14763](#) (2023)
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., & Azhar, F. (2023). Llama: Open and efficient foundation language models [arXiv:2302.13971](#) arXiv preprint.
- Ven, G. M., & Tolia, A. S. (2019). Three scenarios for continual learning [arXiv:1904.07734](#) arXiv preprint.
- Wang, X., Chen, T., Ge, Q., Xia, H., Bao, R., Zheng, R., Zhang, Q., Gui, T., & Huang, X. (2023). Orthogonal subspace learning for language model continual learning [arXiv:2310.14152](#) arXiv preprint.
- Wu, T., Caccia, M., Li, Z., Li, Y.-F., Qi, G., Haffari, G.: Pretrained language model in continual learning: A comparative study. In: *International Conference on Learning Representations* (2021)
- Wu, T., Luo, L., Li, Y.-F., Pan, S., Vu, T.-T., & Haffari, G. (2024). Continual learning for large language models: A survey [arXiv:2402.01364](#) arXiv preprint.
- Wang, A., Pruksachatkun, Y., Nangia, N., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. (2019). Superglue: A stickier benchmark for general-purpose language understanding systems. *Advances in Neural Information Processing Systems*. <https://doi.org/10.48550/arXiv.1905.00537>
- Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. R. (2018). Glue: A multi-task benchmark and analysis platform for natural language understanding [arXiv:1804.07461](#) arXiv preprint.
- Wang, Z., Zhang, Z., Lee, C.-Y., Zhang, H., Sun, R., Ren, X., Su, G., Perot, V., Dy, J., Pfister, T.: Learning to prompt for continual learning. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 139–149 (2022)
- Xia, W., Qin, C., & Hazan, E. (2024). Chain of loRA: Efficient fine-tuning of language models via residual learning. *ICML 2024 Workshop on LLMs and Cognition*
- Yoon, J., Kim, S., Yang, E., & Hwang, S. J. (2020). Scalable and order-robust continual learning with additive parameter decomposition. In: *International Conference on Learning Representations*
- Yoon, J., Yang, E., Lee, J., & Hwang, S. J. (2017). Lifelong learning with dynamically expandable networks [arXiv:1708.01547](#) arXiv preprint.
- Zhang, C., Bengio, S., & Singer, Y. (2022). Are all layers created equal? *Journal of Machine Learning Research*, 23(67), 1–28.

- Zhang, Q., Chen, M., Bukharin, A., He, P., Cheng, Y., Chen, W., Zhao, T.: Adaptive budget allocation for parameter-efficient fine-tuning. In: The Eleventh International Conference on Learning Representations (2023)
- Zaken, E. B., Ravfogel, S., & Goldberg, Y. (2021). Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models [arXiv:2106.10199](https://arxiv.org/abs/2106.10199) arXiv preprint.
- Zhou, D.-W., Sun, H.-L., Ye, H.-J., & Zhan, D.-C. (2024). Expandable subspace ensemble for pre-trained model-based class-incremental learning. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 23554–23564)
- Zhao, W., Wang, S., Hu, Y., Zhao, Y., Qin, B., Zhang, X., Yang, Q., Xu, D., Che, W.: Sapt: A shared attention framework for parameter-efficient continual learning of large language models. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 11641–11661 (2024)
- Zhao, J., Zhang, Y., Chen, B., Schaefer, F. T., & Anandkumar, A. (2023). Incremental low-rank learning. Workshop on Efficient Systems for Foundation Models @ ICML2023
- Zhao, J., Zhang, Z., Chen, B., Wang, Z., Anandkumar, A., & Tian, Y. (2024). Galore: Memory-efficient LLM training by gradient low-rank projection. 5th Workshop on Practical ML for Limited/low Resource Settings
- Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level convolutional networks for text classification. *Advances in Neural Information Processing Systems*. <https://doi.org/10.48550/arXiv.1509.01626>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.